OOP Question

1. What are the four main pillars of Object-Oriented Programming?

- **Encapsulation:** Wrapping data (variables) and code acting on the data (methods) together as a single unit (class), and restricting direct access to some of the object's components (using access modifiers like private).
- **Abstraction:** Hiding complex implementation details and showing only the essential features of an object. In Java, this is achieved using abstract classes and interfaces.
- **Inheritance:** A mechanism where a new class acquires the properties and behaviors of an existing class, promoting code reusability (representing an "IS-A" relationship).
- **Polymorphism:** The ability of an object to take on many forms. It allows methods to do different things based on the object it is acting upon (achieved via method overloading and overriding).

2. What is the difference between Method Overloading and Method Overriding?

- **Method Overloading (Compile-time Polymorphism):** Occurs when multiple methods in the *same class* share the same name but have different parameters (different type, number, or both).
- **Method Overriding (Run-time Polymorphism):** Occurs when a subclass provides a specific implementation for a method that is already defined in its parent class. The method signature must be exactly the same.

Java

```
class MathUtil {
   // Overloading
   int add(int a, int b) { return a + b; }
   int add(int a, int b, int c) { return a + b + c; }
}


class Animal {
   void makeSound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
   // Overriding
   @Override
```

```
    void makeSound() { System.out.println("Bark"); }
}
```

3. What is the difference between an Abstract Class and an Interface in Java?

- **Abstract Class:** Can have both abstract (without implementation) and concrete (with implementation) methods. A class can only extend *one* abstract class. It can have instance variables and constructors.
- **Interface:** Prior to Java 8, could only contain abstract methods (now they can have default and static methods). A class can implement *multiple* interfaces. All variables in an interface are implicitly public static final.

4. Why should we favor "Composition over Inheritance"?

- **Inheritance** creates a rigid, tightly coupled "IS-A" relationship. If the parent class changes, it can break the subclasses. It also limits design since Java does not support multiple inheritance of classes.
- **Composition** relies on a "HAS-A" relationship, where a class contains objects of other classes to provide desired functionality. It promotes loose coupling, greater flexibility, and makes it easier to change behavior at runtime.

5. Explain the static keyword in Java.

- The static keyword indicates that a member (variable or method) belongs to the *class* itself, rather than to instances (objects) of the class.
- **Static Variables:** Shared among all instances of the class. Memory is allocated only once.
- **Static Methods:** Can be called without creating an object of the class (e.g., Math.max()). They cannot access non-static members directly or use the this keyword.

6. What is a Constructor? Can it be overridden?

- A constructor is a special block of code that initializes a newly created object. It has the same name as the class and no return type.
- **Constructors cannot be overridden.** When a subclass is created, it has its own constructors. However, a constructor *can* be overloaded to provide multiple ways to initialize an object.

7. What is the significance of the final keyword?

- **Final Variable:** Its value cannot be changed once initialized (acts as a constant).
- **Final Method:** It cannot be overridden by subclasses. This is useful for preventing unexpected behavior changes in core methods.
- **Final Class:** It cannot be inherited/extended by any other class (e.g., the String class in Java is final).

8. What is the difference between == and .equals() in Java?

- == is an operator that compares the **memory references** of two objects. It checks if both references point to the exact same object in the heap memory.
- .equals() is a method that (if properly overridden) compares the actual **values/state** of the objects for logical equality.

Java

```
String s1 = new String("Google");

String s2 = new String("Google");

System.out.println(s1 == s2);      // False (different memory locations)

System.out.println(s1.equals(s2)); // True (same value)
```

9. How does Garbage Collection work in Java OOP?

- In Java, memory management is handled automatically by the Garbage Collector (GC).
- When an object is no longer referenced by any part of the active program (i.e., it becomes unreachable), the GC automatically destroys it and reclaims the heap memory. Developers do not need to manually free memory like they do in C or C++.

10. What is a Singleton class and how do you implement it?

- A Singleton is a design pattern that ensures a class has only *one* instance in the entire application, and provides a global point of access to it.
- It is implemented by making the constructor private and providing a public static method to return the single instance.

Java

```
public class DatabaseConnection {

   private static DatabaseConnection instance;


   // Private constructor prevents instantiation from other classes

   private DatabaseConnection() {}


   public static DatabaseConnection getInstance() {

      if (instance == null) {

         instance = new DatabaseConnection();

      }
```

```
        return instance;
    }
}
```